Stoquify One-Week Recovery Roadmap Execution Prompt — 2026-08-10

Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Review team

Permanent reviewers:

- Principal enterprise/platform architect: preserve domain ownership, dependency order, tenancy boundaries, modularity, and explicit integration contracts.
- Staff backend/domain and integration engineer: protect server-owned business truth, transactional boundaries, APIs, events, idempotency, concurrency, and provider failure handling.
- Principal data/database and migration architect: protect schema integrity, monetary precision, provenance, retention, backfills, rollback safety, and zero-loss migrations.
- Principal application-security, IAM, privacy, and abuse-resistance architect: enforce tenant isolation, RBAC, entitlement, fresh authentication, segregation of duties, redaction, secrets safety, auditability, and least privilege.
- Senior frontend and design-systems engineer: deliver maintainable, performant, responsive, state-complete surfaces aligned with server contracts.
- Principal workflow/service designer, accessibility specialist, and localization/content strategist: validate end-to-end journeys, dense-workflow ergonomics, WCAG behavior, bilingual copy, recoverability, and human factors.
- Principal product strategist and business-process analyst: connect user outcomes, lifecycle states, operating procedures, prioritization, and measurable product value.
- Principal quality engineer and release-assurance lead: require proportionate unit, integration, contract, migration, browser, accessibility, failure-path, rollback, and evidence-producing release gates.
- Principal SRE/DevSecOps, observability, resilience, and performance engineer: cover deployment, availability, queues/workers, telemetry, alerting, incident recovery, capacity, latency, and cost.

- Principal SaaS platform, packaging, billing, growth, customer-success, and product-operations strategist: protect module lifecycle, entitlement, pricing boundaries, adoption, supportability, and sustainable commercialization.

Because this is a platform-wide recovery roadmap, obtain an explicit finding or `not applicable` decision from:

- Finance, OHADA accounting, treasury, reconciliation, fraud-risk, and internal controls.
- OHADA/SYSCOHADA statutory, tax, payroll, labor, privacy, country-pack, and regulatory compliance.
- Audit, evidence, records governance, and data quality.
- POS, inventory, offline-first, edge-sync, and distributed consistency.
- Purchasing, supplier-risk, AP, maker-checker, and payment controls.
- HRIS, payroll, compensation, attendance, employee privacy, and self-service.
- Payments, mobile-money, provider integration, settlement, suspense, and reconciliation.
- Accounting close, ledger, fiscal documents, reporting, and accountant portal.
- Analytics, BI, metric governance, experimentation, and decision support.
- AI/agent safety, evaluation, human approval, prompt-injection defense, and token/cost governance.
- API, webhook, event/outbox, messaging, import/export, and third-party boundaries.
- Change management, documentation, training, support, rollout, and operational readiness.

Project context

Project: Stoquify.

Workspace: `E:\ohada saas\Focused projects\stoquify`

Primary source report:

`what-next/platform-progress-recovery-assessment-2026-08-10.md`

Mission

Turn the assessment report and current repository state into a realistic, execution-ready seven-day recovery roadmap that produces meaningful, externally verifiable progress.

Do not assume the entire Stoquify platform can be completed within one week. Define precisely what “meaningful progress” can credibly mean within seven calendar days. The weekly roadmap itself must be completely executable within that period.

Decompose only the selected seven-day critical path to minute operational detail. Keep work outside that path at milestone level so this exercise does not become another high-token, low-execution planning cycle.

This is a roadmap and execution-planning run. Do not modify application code, schemas, migrations, configuration, tests, dependencies, or existing evidence artifacts. The only permitted new artifact is:

`what-next/platform-one-week-recovery-roadmap-YYYY-MM-DD.md`

Starting hypothesis

Use this recovery sequence unless current evidence proves that another sequence would produce more verified value:

1. Release Hygiene Greenline.
2. Supplier Workflow Browser Certification.
3. Customer Referral Exact Pilot Release.

Known findings to revalidate, not blindly repeat:

- `npm run error:boundary:fail` previously reported 12 active findings.
- `npm run regulatory:hardcode:fail` previously reported one critical VAT-literal finding.
- TypeScript, Prisma validation, service-boundary, inventory-boundary, hard-delete, demo-trust, workflow-assurance runtime, and receipt-token configuration checks previously passed.
- Supplier workflow completion remains conditional because authenticated tenant and restricted-role browser fixtures and browser evidence are missing.
- Customer referral release scope previously contained nine unclassified paths and four mixed shared files requiring isolation.
- Customer referral pilot evidence previously showed zero of sixteen checks ready because the target environment was not configured.
- Referral production activation still depends on exact revision isolation, security-diff evidence, deployment environment, secrets, HTTPS origin, provider credentials, and pilot-organization entitlement.
- Statutory country-pack readiness remains dependent on qualified expert approval and must not be represented as certified without that evidence.
- The repository contains extensive overlapping reports and readiness artifacts. Do not conduct another indiscriminate scan of all `what-next/` files.

Required work

1. Establish current truth

Inspect the primary assessment and the latest authoritative artifacts underlying its conclusions.

For every relevant finding, classify it as:

- confirmed current;
- completed;
- partially completed;
- genuinely blocked;
- superseded or stale;
- unverified;
- outside the seven-day critical path.

A finding may be marked current only when supported by a file, command result, current repository state, or clearly identified external dependency.

Do not confuse symptoms with root causes, missing evidence with missing implementation, engineering work with external approval, conditional readiness with release readiness, or internally passing controls with an externally usable product journey.

2. Define the seven-day outcome

Write a one-sentence weekly outcome that is concrete and testable.

Define:

- the committed outcome;
- stretch outcomes;
- explicit non-goals;
- the exact user journeys or release capabilities expected to work;
- the evidence required to accept each outcome;
- what will remain incomplete after the week.

The committed outcome must be achievable using the available team, repository state, and dependencies. Do not hide infeasibility behind optimistic language.

3. Produce an exhaustive blocker register

Create a unique blocker ID for every blocker affecting the selected critical path.

For every blocker, state:

- blocker ID and concise name;
- affected outcome and user value;
- present symptom;
- actual root cause;
- current evidence and exact source;
- whether the evidence is current or stale;
- category: engineering, architecture, data, security, testing, environment, provider, regulatory, expert approval, product decision, release isolation, or operational;
- severity and critical-path impact;
- controllable internally or dependent on an external party;
- prerequisite inputs;
- dependent tasks;
- owner role;
- exact next action;
- files or systems expected to change during execution;
- verification command or acceptance scenario;
- evidence artifact required for closure;
- earliest realistic completion;
- workaround, if one exists;
- decision or approval deadline;
- escalation condition;
- consequence if not resolved;
- status: open, ready, in progress, blocked, resolved, deferred, or removed as stale.

Do not call work “blocked” merely because it is difficult. A genuine blocker must require missing authority, information, access, infrastructure, credentials, expert approval, or external-state change.

4. Identify all conditions required for progress

Build a conditions-of-success register covering:

- scope freeze and work-in-progress limits;
- owners and daily availability;
- user decisions and approval deadlines;
- authenticated tenant and restricted-role fixtures;
- database and non-production environment availability;
- secrets, HTTPS origin, and provider credentials;
- exact release revision and clean scope isolation;
- test data and migration state;
- browser, responsive, accessibility, and failure-path tooling;
- security-diff and graph refresh requirements;
- regulatory or accounting expert review;
- release-gate prerequisites;
- operational rehearsal and rollback readiness;
- token and execution-efficiency tracking.

For every condition, give the required state, current state, gap, owner, deadline, validation method, and effect on the critical path if missed.

5. Build the dependency graph and critical path

Represent the selected work as a dependency graph.

Identify:

- tasks that can start immediately;
- tasks that can run concurrently;
- sequential dependencies;
- external decision gates;
- the true critical path;
- tasks that should be removed from this week;
- the earliest credible completion date.

Use the report's recommended sequence as the default hypothesis, but change it if repository evidence shows that another sequence unlocks more verified product value within seven days.

6. Create the detailed execution board

Decompose the seven-day critical path into tasks no larger than approximately half a working day. Avoid artificial granularity when a task cannot be meaningfully divided.

For every task, specify:

- task ID;
- outcome;
- owner role;
- estimated duration;
- prerequisite task or decision;
- exact scope and likely files;

- actions to perform;
- acceptance criteria;
- focused test or verification command;
- evidence to retain;
- rollback or recovery requirement;
- stop condition;
- next task unlocked.

Organize the execution board by exact calendar date and morning/afternoon checkpoint.

Use this candidate schedule as a starting hypothesis:

- Day 1: freeze scope, revalidate current truth, consolidate user decisions, and establish the execution board.
- Days 1–3: clear release-hygiene failures and make the selected baseline gates green.
- Days 3–5: establish supplier fixtures and complete authenticated, restricted-role, responsive, accessibility, and export-redaction browser verification.
- Days 5–7: isolate the customer-referral release scope, create or specify the exact revision, regenerate graph evidence, run security-diff verification, and rehearse the release in non-production if the environment is supplied.
- Day 7: conduct evidence review, go/no-go decision, and publish the next bounded execution slice.

Adjust the schedule only when the evidence justifies it.

7. Define measurable progress

Establish a baseline, daily target, and end-of-week target for:

- critical-path tasks completed;
- blockers resolved;
- blockers awaiting external action;
- release gates passing;
- verified user journeys;
- acceptance scenarios passed;
- unresolved high-severity findings;
- work-in-progress count;
- rework cycles;
- generated planning/readiness artifacts;
- token consumption per accepted deliverable, when actual token telemetry is available.

If actual token totals are unavailable, say so. Do not invent token data. Use session count, elapsed effort, command runs, changed files, closed blockers, and accepted deliverables as secondary indicators until proper telemetry exists.

Operating limits:

- Maximum active work: one implementation slice and one unblocker.
- No new product workstreams during the seven-day recovery period.

- No repeated whole-repository audit unless material repository changes invalidate the existing assessment.
- No new readiness report unless it replaces a canonical report or closes a named blocker.
- Stop and escalate after two unsuccessful attempts that produce no new evidence.
- A task is complete only when its acceptance criteria and evidence pass.
- Keep planning and status narration subordinate to implementation and verification.

8. Define verification and release acceptance

For each planned implementation slice, name the minimum verification chain.

At minimum, account for these commands where relevant:

```
git status --short --branch
npm run prisma:validate
npm run typecheck
npm run error:boundary:fail
npm run regulatory:hardcode:fail
npm run service:boundary:fail
npm run inventory:boundary:fail
npm run hard-delete:fail
npm run demo:trust:fail
npm run workflow:assurance:runtime-check
npm run receipt:token:config-gate
```

Also identify:

- focused unit and integration tests;
- authenticated browser scenarios;
- restricted-role denial scenarios;
- export-redaction checks;
- responsive and accessibility checks;
- security-diff verification;
- exact-revision and clean-worktree verification;
- non-production release rehearsal;
- rollback and failure-path verification.

Do not prescribe broad `policy:gates` or `build:app` runs repeatedly. Schedule them only where they provide release-decision evidence.

9. Create a consolidated decision packet

Identify every decision or authorization required from the user.

For each decision, provide:

- exact question;
- available options;
- recommended option;

- evidence supporting the recommendation;
- decision deadline;
- work that can continue while waiting;
- consequence of no decision.

Consolidate the decisions into one Day-1 packet wherever possible. Do not interrupt execution repeatedly for decisions that could have been identified upfront.

10. Define daily governance

Specify a concise daily checkpoint containing only:

- completed and accepted tasks;
- failed acceptance checks;
- blockers opened or closed;
- external decisions overdue;
- critical-path change;
- actual versus planned progress;
- token or effort efficiency, if measurable;
- next day's committed outcome.

Require the same canonical roadmap to be updated during future execution. Do not generate a new status document every day.

Required report structure

1. Executive feasibility verdict.
2. Definition of meaningful progress for the seven-day window.
3. Current-truth register.
4. Completed, partial, blocked, stale, and unverified work.
5. Exhaustive blocker register.
6. Conditions-of-success register.
7. Dependency graph and critical path.
8. Seven-day calendar and half-day execution board.
9. Task-level definitions of ready and done.
10. Verification and evidence matrix.
11. User decision packet.
12. Progress metrics and token-efficiency controls.
13. Risk, rollback, and escalation register.
14. End-of-week go/no-go criteria.
15. Post-week backlog at milestone level only.
16. The three actions that must begin immediately.

Evidence to inspect

- `what-next/platform-progress-recovery-assessment-2026-08-10.md`

- Latest supplier workflow review and completion reports.
- Latest customer workflow completion report.
- `what-next/referrals/REFERRAL_WAR_ROOM_STATUS.md`
- `what-next/referrals/CUSTOMER_REFERRAL_TAKEOFF_READINESS_2026-08-09.md`
- `what-next/referrals/customer-referral-release-scope-readiness.md`
- `what-next/referrals/customer-referral-pilot-evidence-readiness.md`
- `what-next/statutory-country-pack-production-readiness.md`
- Current gate reports directly related to the selected critical path.
- `package.json` and `.github/workflows/ci.yml`.
- Current Git status and recent relevant history.
- Relevant sources in `services/`, `actions/`, `app/`, `components/`, `scripts/`, `prisma/`, and focused tests.
- Relevant `graphify-out/` reports and graph data for dependency and impact analysis.

Use targeted searches. Do not read thousands of historical artifacts when current canonical evidence already exists.

Risk controls

- Preserve dirty-worktree changes and do not overwrite unrelated user work.
- Do not modify code, schema, migrations, dependencies, configuration, tests, or existing reports during this planning run.
- Do not propose a broad redesign without evidence that the current architecture prevents the seven-day outcome.
- Do not add speculative features.
- Preserve service-owned business truth, tenant isolation, RBAC, module entitlement, maker-checker controls, audit evidence, redaction, and safe errors.
- Do not treat an external approval as an engineering implementation task.
- Do not claim release, legal, tax, accounting, security, privacy, or accessibility certification without the required evidence.
- Do not use report volume, line count, file count, or test count alone as evidence of product value.
- Do not hide unresolved blockers in optimistic completion percentages.

Success criteria

- One credible, bounded, seven-day outcome is selected.
- Every critical-path blocker has an owner, action, dependency, deadline, and closure test.
- Every task scheduled for the week is small enough to execute and independently verify.
- External decisions and prerequisites are consolidated and deadline-bound.
- The roadmap distinguishes committed work from stretch work.
- The plan limits WIP, repeated discovery, report generation, and low-value token consumption.
- Progress is measurable daily through accepted deliverables and closed blockers.
- End-of-week go/no-go criteria are binary and evidence-based.
- The roadmap fits in one canonical report.

- The final report ends with exactly three immediate actions, each naming an owner, deliverable, deadline, and acceptance criterion.

Non-goals

- Completing every Stoquify module in one week.
- Reopening completed internal-control slices without contradicting evidence.
- Starting new HRIS, payroll, POS, inventory, module-commercialization, or country-pack initiatives unless proven critical-path dependencies.
- Conducting another general platform audit.
- Producing additional strategy, readiness, or status documents.
- Broad architectural refactoring.
- Unrelated lint cleanup.
- Production deployment without the required authority and target-environment evidence.

Chat response

Return only:

- the feasibility verdict;
- the committed seven-day outcome;
- the critical path;
- the number of confirmed blockers by category;
- the decisions required from the user on Day 1;
- the three immediate actions;
- a link to the saved roadmap.